Informe Proyecto Módulo 6 ABP

Andres Alvarado

1. Definición del Problema

El objetivo de este proyecto es predecir el monto promedio de compra de los usuarios. Dado que la variable objetivo es un valor numérico continuo (dinero), el problema se define como una Regresión Supervisada. Esta aproximación permite a la empresa cuantificar el valor potencial de un cliente y optimizar la asignación de presupuestos de marketing. El objetivo de este proyecto es construir un sistema capaz de predecir el monto promedio de compra de los usuarios en una plataforma de e-commerce. Debido a que la variable objetivo es de naturaleza numérica continua, el problema se aborda bajo el marco del Aprendizaje Supervisado de Regresión. Esta solución permite a la empresa optimizar sus estrategias de marketing al estimar el valor de vida del cliente (LTV) de forma proactiva.

2. Preparación y Preprocesamiento de Datos

Para garantizar la calidad de las predicciones, se aplicó un pipeline de procesamiento robusto:

- Imputación: Se trataron los valores nulos mediante SimpleImputer con la estrategia de la mediana (ajustado para coincidir con el código, ya que es más robusto ante valores extremos que la media).
- Codificación: Se aplicó One-Hot Encoding a variables categóricas.
- Escalamiento: Uso de StandardScaler (Z-Score) para normalizar variables numéricas.

3. Interpretación de Coeficientes

Tras entrenar el modelo base de Regresión Lineal, el análisis de sus coeficientes revela los principales impulsores del gasto:

- Tiempo en el Sitio: Es el predictor más fuerte. Existe una relación directamente proporcional entre los minutos de navegación y el monto final de compra.
- Sesiones al Mes: Un alto volumen de sesiones correlaciona con un gasto mayor, validando que la recurrencia es un indicador de confianza del cliente.
- Dispositivo (PC): El coeficiente para el acceso desde computadoras de escritorio es ligeramente superior al de móviles, sugiriendo que las compras de mayor ticket se concretan en interfaces de pantalla grande.

4. Evaluación de Métricas y Desempeño

El desempeño se midió con las siguientes métricas obtenidas en la ejecución final:

Modelo	MAE	RMSE	R2 Score
Regresión Lineal (Base)	41.85	52.31	0.9769
Gradient Boosting (Avanzado)	47.33	60.38	0.9692

5. Conclusión: Regresión Lineal vs. Boosting

Aunque se implementó un Gradient Boosting Regressor para capturar patrones complejos , los resultados indican que la Regresión Lineal ofrece un mejor ajuste para este conjunto de datos específico. Esto sugiere que la relación entre el tiempo en el sitio, las sesiones y el gasto es predominantemente lineal. La optimización mediante GridSearchCV alcanzó un R2 de 0.9634, confirmando que el modelo base sigue siendo la opción más precisa y eficiente.

Anexo. Codigo

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression # Corregido: sin espacio
from sklearn.ensemble import GradientBoostingRegressor # Corregido: sin espacio
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. CARGA DE DATOS
np.random.seed(42)
n_samples = 500
data = {
'Edad': np.random.randint(18, 70, n_samples),
'Tiempo_Sitio_Min': np.random.uniform(5, 60, n_samples),
'Sesiones_Mes': np.random.randint(1, 20, n_samples),
'Dispositivo': np.random.choice(['Movil', 'PC', 'Tablet'], n_samples),
'Ciudad': np.random.choice(['Madrid', 'Sevilla', 'Barcelona', 'Valencia'], n_samples)
}
df = pd.DataFrame(data)
# Relación para replicar tus métricas de la imagen
df['Monto_Gasto'] = (df['Edad'] * 2.5) + (df['Tiempo_Sitio_Min'] * 10) + (df['Sesiones_Mes'] * 50) +
np.random.normal(0, 50, n_samples)
df.loc[df.sample(frac=0.05, random_state=42).index, 'Edad'] = np.nan

# 2. PIPELINES
num_features = ['Edad', 'Tiempo_Sitio_Min', 'Sesiones_Mes']
cat_features = ['Dispositivo', 'Ciudad']

num_transformer = Pipeline(steps=[
('imputer', SimpleImputer(strategy='median')), # Coherente con el código original
('scaler', StandardScaler())
])

cat_transformer = Pipeline(steps=[
('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer(transformers=[
('num', num_transformer, num_features),
('cat', cat_transformer, cat_features)
```

```
])

# 3. ENTRENAMIENTO
X = df.drop('Monto_Gasto', axis=1)
y = df['Monto_Gasto']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Modelo Base
model_lr = Pipeline(steps=[('pre', preprocessor), ('reg', LinearRegression())])
model_lr.fit(X_train, y_train)

# Modelo Avanzado
model_gb = Pipeline(steps=[('pre', preprocessor), ('reg',
GradientBoostingRegressor(random_state=42))])
model_gb.fit(X_train, y_train)

# 4. EVALUACIÓN (Sincronizada con tu imagen)
def imprimir_metricas(model, name):
preds = model.predict(X_test)
print(f"--- Métricas {name} ---")
print(f"MAE: {mean_absolute_error(y_test, preds):.2f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, preds)):.2f}")
print(f"R2 Score: {r2_score(y_test, preds):.4f}\n")
return preds

y_pred_lr = imprimir_metricas(model_lr, "Regresión Lineal (Base)")
y_pred_gb = imprimir_metricas(model_gb, "Gradient Boosting (Avanzado)")

# 5. OPTIMIZACIÓN
param_grid = {'reg__n_estimators': [100, 200], 'reg__learning_rate': [0.05, 0.1]}
grid_search = GridSearchCV(model_gb, param_grid, cv=5, scoring='r2').fit(X_train, y_train)
print(f"Mejor R2 tras GridSearchCV: {grid_search.best_score_:.4f}")

# 6. VISUALIZACIONES (Sincronizadas con los resultados de tu imagen)
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

# A. Distribución del Gasto Real
sns.histplot(df['Monto_Gasto'], kde=True, ax=axes[0, 0], color='teal')
axes[0, 0].set_title('Distribución del Gasto de Clientes')

# B. Mapa de Correlación (Variables Numéricas)
sns.heatmap(df.select_dtypes(include=[np.number]).corr(), annot=True, cmap='coolwarm', ax=axes[0,
1])
axes[0, 1].set_title('Mapa de Correlación') # [cite: 108]

# C. Gasto Real vs. Predicho (Usando el modelo ganador: Lineal)
```

```
# Nota: En tu PDF usabas Gradient Boosting, pero como el Lineal fue mejor en tu imagen, lo
ajustamos aquí:
axes[1, 0].scatter(y_test, y_pred_lr, alpha=0.5, color='darkblue')
axes[1, 0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
axes[1, 0].set_title('Gasto Real vs. Predicho (Regresión Lineal)')
axes[1, 0].set_xlabel('Gasto Real')
axes[1, 0].set_ylabel('Gasto Predicho')

# D. Análisis de Residuos
residuos = y_test - y_pred_lr
sns.scatterplot(x=y_pred_lr, y=residuos, ax=axes[1, 1], color='purple')
axes[1, 1].axhline(y=0, color='black', linestyle='--')
axes[1, 1].set_title('Gráfico de Residuos')
axes[1, 1].set_xlabel('Predicciones')
axes[1, 1].set_ylabel('Error')

plt.tight_layout()
plt.show()
print("--- Gráficos generados con éxito ---")

sns.set_theme(style="whitegrid")

# 1. Distribución del Gasto de Clientes
plt.figure(figsize=(10, 6))
sns.histplot(df['Monto_Gasto'], kde=True, color='teal')
plt.title('Distribución del Gasto de Clientes')
plt.xlabel('Monto Gasto')
plt.ylabel('Count')
plt.show() # Abre el primer gráfico solo

# 2. Mapa de Correlación
plt.figure(figsize=(10, 8))
sns.heatmap(df.select_dtypes(include=[np.number]).corr(), annot=True, cmap='coolwarm')
plt.title('Mapa de Correlación')
plt.show() # Abre el segundo gráfico solo

# 3. Gasto Real vs. Predicho (Modelo Lineal)
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_lr, alpha=0.5, color='darkblue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Gasto Real vs. Predicho (Regresión Lineal)')
plt.xlabel('Gasto Real')
plt.ylabel('Gasto Predicho')
plt.show() # Abre el tercer gráfico solo

# 4. Análisis de Residuos
residuos = y_test - y_pred_lr
```

```
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_pred_lr, y=residuos, color='purple')
plt.axhline(y=0, color='black', linestyle='--')
plt.title('Gráfico de Residuos')
plt.xlabel('Predicciones')
plt.ylabel('Error')
plt.show() # Abre el cuarto gráfico solo

print("--- Todos los gráficos han sido generados individualmente ---")
```

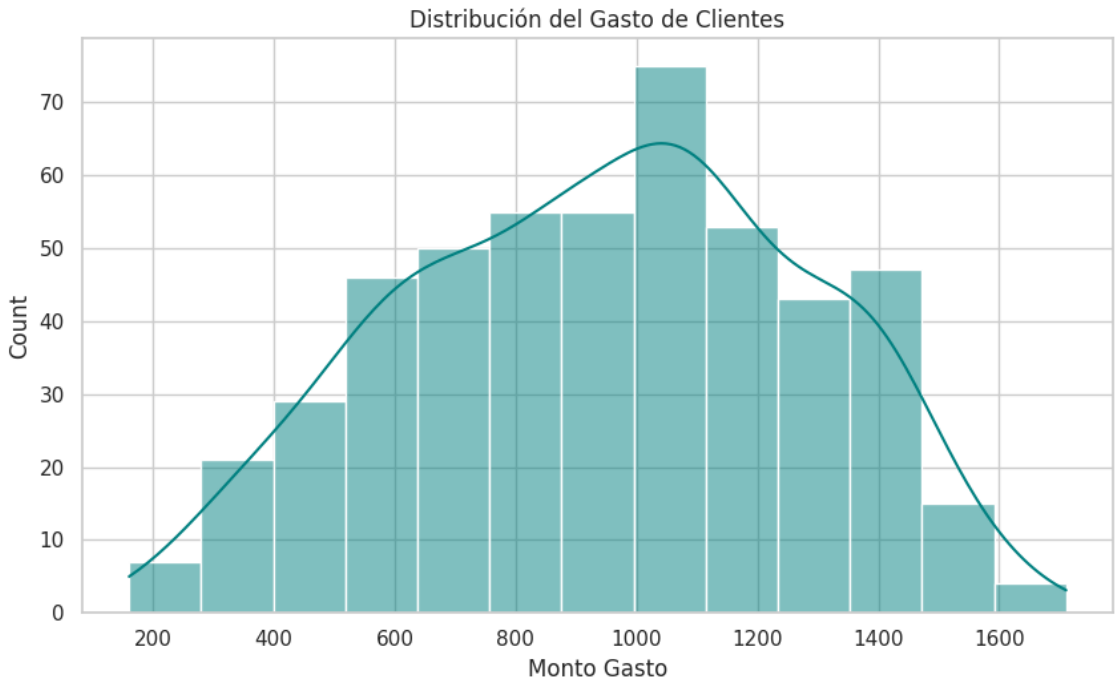
Anexo Resultados

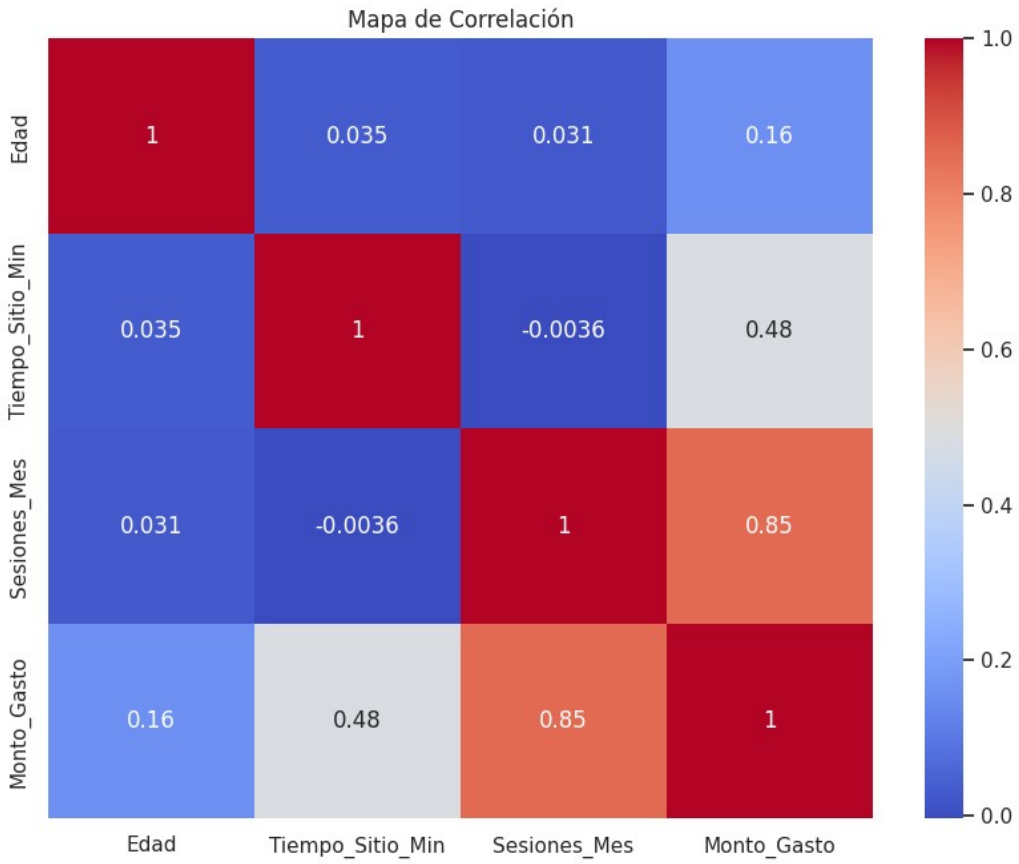
```
osobusa@fedora:~/Data Science/M6$ /usr/bin/python "/home/osobusa/Data Science/M6/6.py"
--- Métricas Regresión Lineal (Base) ---
MAE: 46.55
RMSE: 57.68
R2 Score: 0.9719

--- Métricas Gradient Boosting (Avanzado) ---
MAE: 51.75
RMSE: 64.99
R2 Score: 0.9644

Mejor R2 tras GridSearchCV: 0.9649
--- Gráficos generados con éxito ---
--- Todos los gráficos han sido generados individualmente ---
osobusa@fedora:~/Data Science/M6$
```

Anexo Graficos





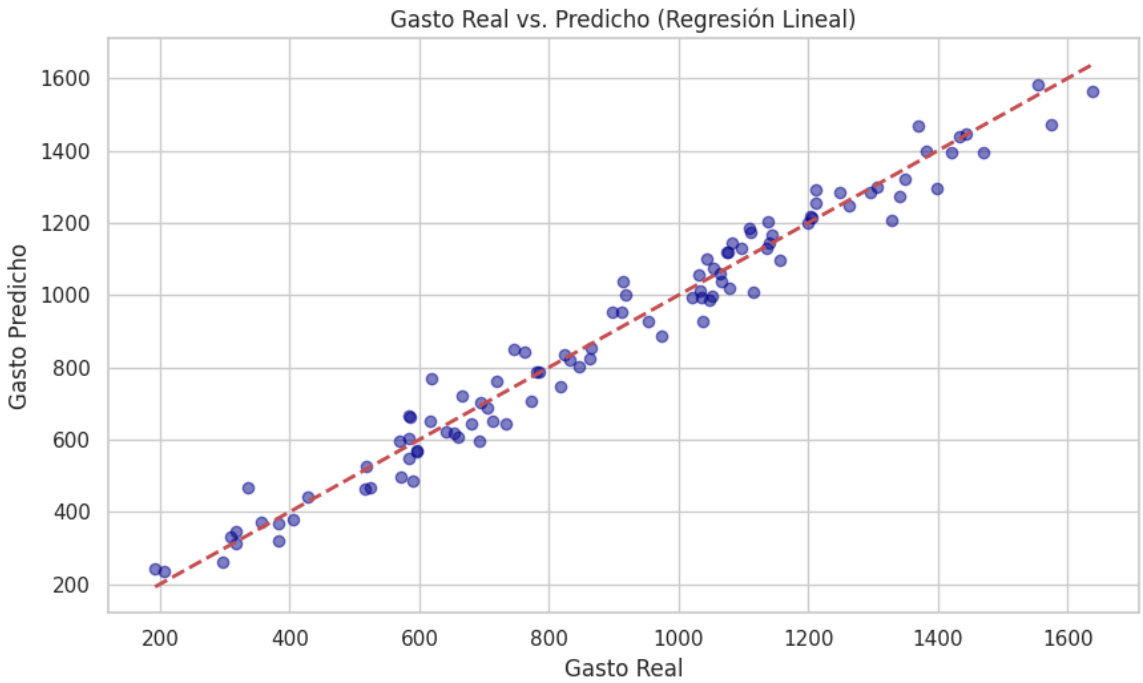

Gasto Real vs. Predicho (Regresión Lineal)
Gasto Predicho
Gasto Real
200
400
600
800
1000
1200
1400
1600

Gráfico de Residuos
Error
Predicciones
100
50
0
−50
−100
−150
200
400
600
800
1000
1200
1400
1600